TECH BIZ SECURITY · SOC ANALYST INTERNSHIP · BATCH B-503

WEEK 3 · TASK 9

Network Traffic Analysis with Wireshark

Submitted by	**Oscar ALIDJINOU**
Student ID	**TBS-INT-2026-2398**
Program	**SOC Analyst Internship (6 weeks)**
Company	**Tech Biz Security**
Date of submission	**03-11-2026**

01 Objective

Logs tell me what a system says happened; network traffic shows what actually crossed the wire. Wireshark is the standard tool for reading that traffic, so this task was about learning to use it: capturing packets, narrowing them with display filters, and dissecting a single packet down to its bytes. I worked through the TryHackMe room Wireshark: The Basics and then captured and analysed real traffic from my own computer.

02 Proof of Completion

Room / activity	Duration	Tasks	Status
TryHackMe: Wireshark: The Basics	60 min	6	Completed, room progress 100%
Live capture on my own Wi-Fi interface	-	-	Done, over 21,000 packets (sections 04 to 06)

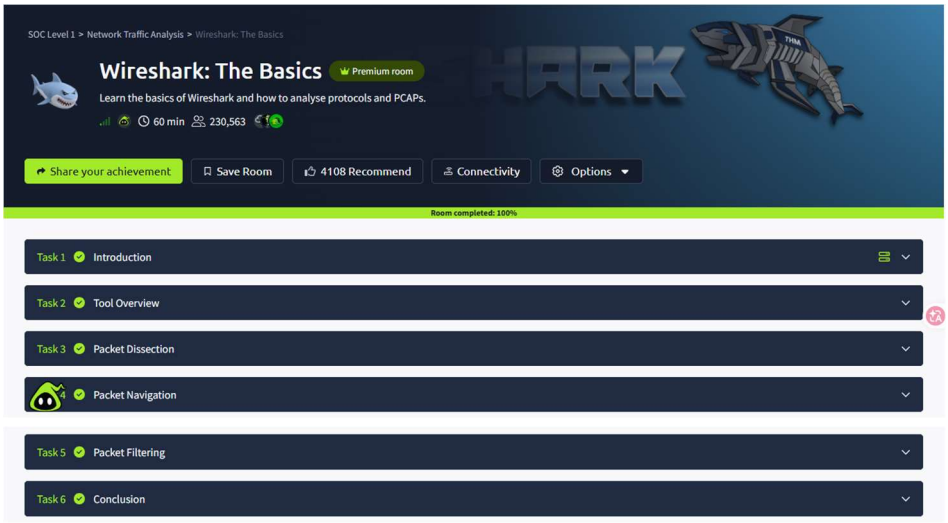


Figure 1. Wireshark: The Basics: room completed (100%).

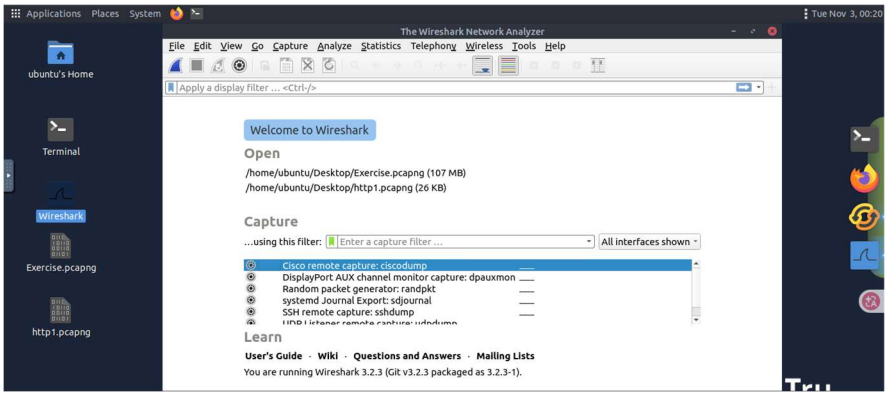


Figure 2. Wireshark open in the room's virtual machine, with the practice capture files.

> **Privacy and legality**
> I captured only the traffic of my own computer on my own network. A capture file holds IP addresses, hardware (MAC) addresses and sometimes passwords or cookies, so in my screenshots I hid every MAC address and the adapter identifier. Capturing traffic on a network I do not own or administer, without permission, is illegal.

03 Wireshark Essentials

Wireshark is a free, open-source packet analyser that works on live traffic or on saved capture files (PCAP). Analysts use it to troubleshoot network problems such as congestion, to spot security problems such as rogue hosts, unusual ports and suspicious traffic, and to study the details of a protocol.

3.1 The three panes

Pane	What it shows
Packet list (top)	One row per packet: number, time, source, destination, protocol, length and a summary.
Packet details (bottom left)	The selected packet split into layers that can be expanded one by one.
Packet bytes (bottom right)	The raw bytes in hexadecimal, with the matching text beside them.

3.2 Packet dissection: the layers of a packet

Wireshark decodes each packet layer by layer, which matches the OSI model I studied in Task 2:

Section in the details pane	Layer	What it shows
Frame	Physical (1)	The packet number, its size in bytes and the capture time.
Ethernet II	Data link (2)	Source and destination MAC addresses.
Internet Protocol	Network (3)	Source and destination IP addresses, TTL and the protocol carried.
TCP or UDP	Transport (4)	Ports, flags and sequence numbers. TCP also reports segments that had to be reassembled.
HTTP, DNS, SMB...	Application (5 to 7)	The fields of the protocol itself.

3.3 Navigating a large capture

Feature	What it does	Why an analyst uses it
Packet numbers and Go to Packet	Every packet has a unique number; I can jump to any of them.	Return to an event of interest in a huge file.
Find Packet (Edit menu)	Searches packets by their content.	Look for an attack pattern or an error string.
Mark packets	Highlights packets with a mark.	Flag packets of interest and export only those.
Packet comments	Notes attached to a packet and stored in the file.	Leave information for a colleague.
Export packets and objects	Saves a subset of packets, or files carried by HTTP, SMB, IMF, TFTP and DICOM.	Recover a transferred file for investigation.
Time Display Format	Changes how time is shown; UTC is the common choice.	Match packets to log entries.
Expert Information	Lists protocol anomalies by severity: Chat (blue), Note (cyan), Warn (yellow), Error (red).	Find malformed packets and other problems quickly.

04 Capturing Traffic on My Own Computer

I opened Wireshark (version 4.6.9) and chose the Wi-Fi interface, the one carrying my real traffic. The start screen lists every interface available, including the loopback adapter, Ethernet and the virtual adapters of my virtual machines. The small line beside Wi-Fi is a live graph of its activity.

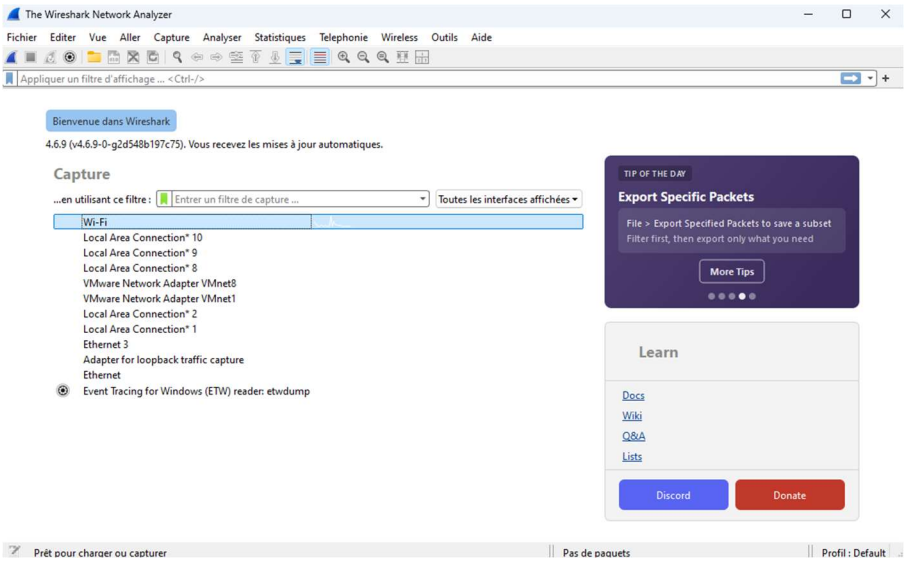


Figure 3. Choosing the Wi-Fi interface to capture.

I then started the capture, browsed normally for a short while, and stopped and saved it as a .pcapng file. The first screenshot shows the capture running after about a minute, with 755 packets.

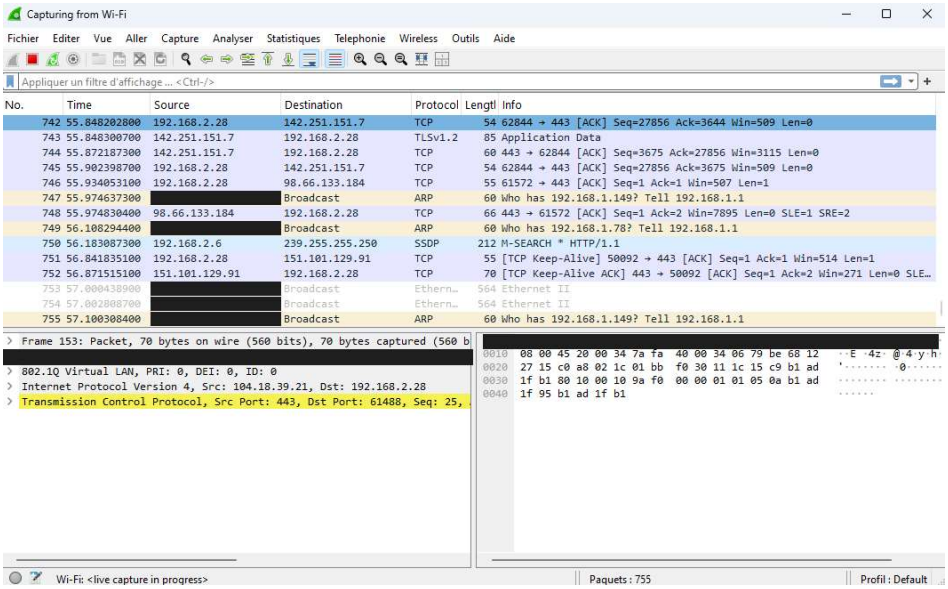


Figure 4. Live capture on the Wi-Fi interface (755 packets). Hardware addresses are hidden.

What I saw	What it is	Normal or suspicious?
TCP [ACK] and Keep-Alive on port 443	Established HTTPS connections, with small packets that keep them open.	Normal background activity.
TLSv1.2 "Application Data"	The encrypted content of an HTTPS session.	Normal. I cannot read it, which is the purpose of encryption.
ARP "Who has 192.168.1.149? Tell 192.168.1.1"	A broadcast from the router asking which device owns an address.	Normal on a local network. Many replies from different devices for one address would be a warning sign.
SSDP "M-SEARCH" to 239.255.255.250	A device looking for others on the network (UPnP discovery).	Normal for printers, TVs and similar devices.

> **What this tells me**
> Almost everything on my connection was encrypted on port 443. A defender still sees who talks to whom, when, how often and how much, even when the content is hidden. That metadata is often enough to spot a problem.

05 Display Filters

A busy capture holds thousands of packets, so filters are essential. There are two kinds: a capture filter limits what is recorded, while a display filter hides packets that are already captured and leaves only the ones I want to see. A filter bar that turns green means the syntax is valid; red means it is not.

Filter	What it shows	Used in my capture
`dns`	Only DNS packets.	Yes (Figure 5)
`ip.src == 192.168.2.28`	Only packets sent by my computer.	Yes (Figure 6)
`http`	Only HTTP packets (unencrypted web traffic).	Reference
`ip.addr == x.x.x.x`	Every packet to or from one address.	Reference

Filter	What it shows	Used in my capture
`tcp.port == 443`	Traffic on a TCP port.	Reference
`tcp.flags.reset == 1`	TCP reset packets.	Reference
`arp`	Only ARP packets.	Reference
`http.request.method == "POST"`	Web forms sent to a server.	Reference
`frame contains "password"`	Any packet whose bytes contain a word.	Reference

Right-click options are also useful: Apply as Filter and Prepare as Filter build a filter from a field I click, Conversation Filter and Colourise Conversation isolate or highlight one conversation, Apply as Column adds a field to the packet list, and Follow Stream rebuilds a whole conversation in one window.

5.1 Filtering DNS

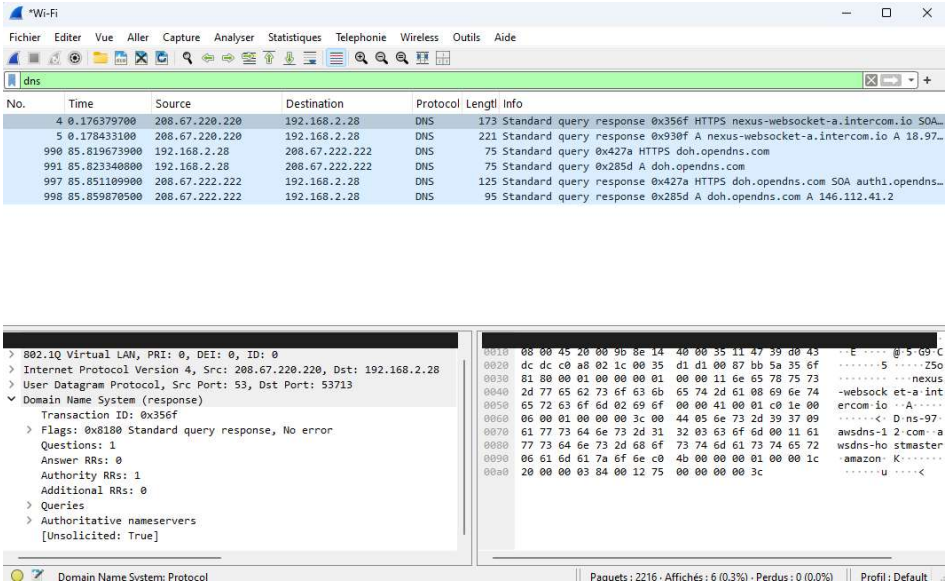


Figure 5. The dns filter: 6 of 2,216 packets.

Only 6 of the 2,216 packets (0.3 %) were DNS. They show two names being looked up: doh.opendns.com, whose answer was the address 146.112.41.2, and an Intercom chat service. All answers came from servers belonging to OpenDNS, which means my computer uses OpenDNS as its DNS service.

5.2 Filtering by source address

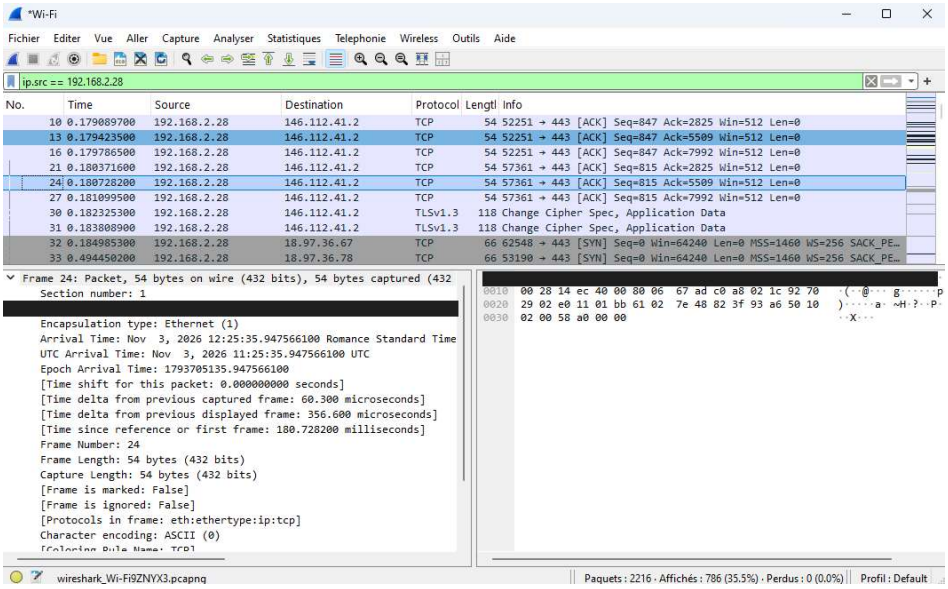


Figure 6. ip.src == 192.168.2.28: 786 of 2,216 packets (35.5 %).

This filter kept only what my own computer sent, which was about a third of the capture. Many of these packets are sent to 146.112.41.2 on port 443, the same address that DNS returned for doh.opendns.com. The traffic then carries TLS 1.3 handshake messages, so I can follow the whole chain: a name is looked up, an address is returned, and my computer opens an encrypted connection to it. This pattern is consistent with DNS over HTTPS, a setting where the browser sends its DNS questions inside HTTPS.

> **Why that matters to a SOC**
> When DNS travels over HTTPS, the dns filter no longer shows those lookups. Defenders who rely on plain DNS logs to find suspicious domains can lose visibility, so they have to control or monitor DNS over HTTPS as well.

06 Analysis of One Packet

6.1 Frame 24: a TCP acknowledgement

I selected packet 24, highlighted in Figure 6, and read it layer by layer.

Layer	Field and value	Meaning
Frame	Frame 24, 54 bytes on the wire. Arrived on 3 Nov 2026 at 12:25:35 local time (11:25:35 UTC).	The packet is 14 bytes of Ethernet, 20 of IP and 20 of TCP, with no data.
Ethernet II	Source: my computer; destination: my router (addresses hidden).	The packet is leaving my computer for the router, the first hop on the way to the internet.
IPv4	192.168.2.28 to 146.112.41.2, TTL 128, protocol TCP, Don't Fragment set.	A private address talking to a public one. A TTL of 128 is typical of Windows.
TCP	Port 57361 to port 443, flag ACK, Seq 815, Ack 2825, window 512, length 0.	A pure acknowledgement inside an existing HTTPS session. Port 443 is HTTPS; 57361 is a temporary port chosen by my computer.

6.2 Reading the raw bytes

The packet bytes pane shows the same information in hexadecimal. Decoding the main fields by hand proved that I understood where each value comes from:

Bytes (hex)	Field	Value
45	IP version and header length	Version 4, header of 20 bytes
00 28	Total length	40 bytes (20 IP + 20 TCP)
40 00	Flags and fragment offset	Don't Fragment is set
80	Time to live	128
06	Protocol	6 means TCP
c0 a8 02 1c	Source IP address	192.168.2.28
92 70 29 02	Destination IP address	146.112.41.2
e0 11	Source port	57361
01 bb	Destination port	443
50 10	TCP header length and flags	20 bytes, ACK flag set
02 00	Window size	512

> **Verdict**
> Benign. The destination is the address my DNS lookup returned, the port is the standard HTTPS port, the packet carries no data, and it belongs to a normal, ongoing TLS session.

6.3 A second packet: a TCP reset

In a later, longer capture of 21,123 packets, Wireshark highlighted one packet in dark red: number 21116, a TCP packet with the RST flag. Wireshark colours resets so that they stand out.

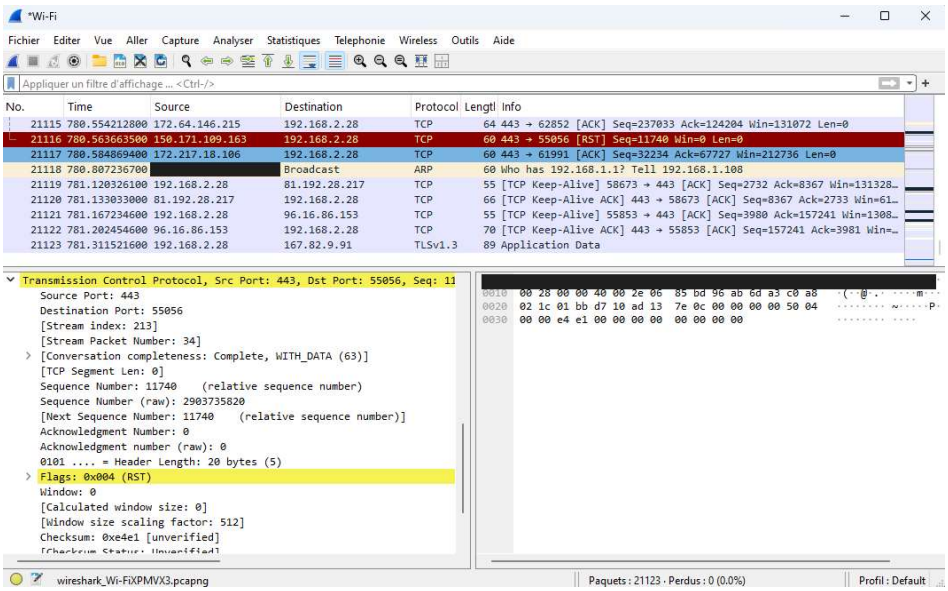


Figure 7. A TCP reset (packet 21116) in a capture of 21,123 packets.

Field	Value and meaning
Direction	From a server on port 443 to my computer on port 55056.
Flags	0x004: RST. The connection is being cut off immediately instead of closed politely.
Window and length	Window 0, length 0: no data, nothing left to send.
Conversation completeness	Complete, with data. The connection had a full handshake and had carried data, so it was a real session.
Verdict	Benign. Servers commonly reset idle connections. It becomes interesting only in numbers: many resets to many ports from one host can reveal a port scan.

07 Normal vs Suspicious Traffic

Feature	Normal (in my capture)	Suspicious
Web traffic	Almost all HTTPS (443) with encrypted data.	Plain HTTP carrying credentials or sensitive data.
Connections	A few long-lived connections to well-known services.	Many SYN packets to many ports from one host (a port scan).
ARP	A router asking who owns an address.	Two different hardware addresses claiming one IP address (ARP spoofing).
DNS	A few lookups of readable names.	Very long, random-looking names, or a flood of lookups to one domain.
Volume	Small packets and acknowledgements.	Large, steady outbound transfers to an unfamiliar address.
Ports	443, 53, 80.	Unusual ports such as 4444 used for remote control.

What a plaintext password looks like

If a site uses plain HTTP, a login form is sent as readable text, and anyone able to capture the traffic can read it. The example below is illustrative and uses made-up values; it is not from my capture.

```
POST /login HTTP/1.1
Host: shop.example
Content-Type: application/x-www-form-urlencoded

username=alice&password=Summer2026!
```

To find such packets, I would filter on http.request.method == "POST" or frame contains "password", then right-click the packet and choose Follow HTTP Stream to read the whole exchange. In my own capture nothing like this appeared, because my traffic was encrypted. That is exactly why HTTPS matters.

Link to ATT&CK

Technique	Name	What Wireshark shows
T1040	Network Sniffing	What an attacker gains by capturing traffic on a network, as I did with my own.
T1046	Network Service Discovery	Many connection attempts across ports from one host.
T1557.002	ARP Cache Poisoning	ARP replies that tie a known IP address to a new hardware address.
T1071.004	Application Layer Protocol: DNS	Unusual DNS names or volumes used for command and control.

08 Reflection and Key Takeaways

- Wireshark turns invisible traffic into something I can read, layer by layer, from the Ethernet frame up to the application.
- Filters are what make a capture usable: 786 packets out of 2,216 and 6 out of 2,216 are far easier to understand than the whole file.
- Encryption protects content but not metadata, so IPs, ports, timing and DNS still tell a story.
- A capture file is sensitive data. I hid hardware addresses before sharing screenshots, and I will treat real capture files with care.

Next step: practise on malicious sample captures, following streams and exporting objects to find what an attacker did.

09 References

- TryHackMe: Wireshark: The Basics (room)
- Wireshark User's Guide: display filters and packet dissection
- MITRE ATT&CK: T1040, T1046, T1557.002 and T1071.004